

- TestCase
- LiveServerTestCase

Besides these classes, you can always write unique test codes for checking specific functions and pieces of the system. The best feature in a Django framework is the execution of the assert method. It helps the testing results to appear in a certain parameter of True or False. When you begin testing the framework, it will execute the tests set by you with the respected classes. These test execution methods may work independently but within a similar setup with predefined classes. Here is an example:

```

• class MyTestClass(TestCase):
•     def setUp(self):
•         # Setup runs before every second test method.
•         pass
•
•     def tearDown(self):
•         # Clean up runs after every third test method.
•         pass
•
•     def test_something_that_will_pass(self):
•         self.assertFalse(False)
•
•     def test_something_that_will_fail(self):
•         self.assertTrue(False)

```

If you are new to testing Django apps, the best class to use is `django.test.TestCase`. It is a test class that helps developers create a cleaner text code with a useful database. The test runs smoothly, executing every transaction of the test function. It also provides a test client to enable the simulation of user interactions with the system code at the representation layer.

However, there is some issue with the usage of the `django.test.TestCase`. The test run can sometimes result in a slower process. What happens is some tests need a measurement of the user interaction at the view-level, so the same setup will slowdown other test runs. You will need extensive knowledge of test classes; you can easily replace them with types that don't need simulations.

Here, we will learn more about unit testing for Django apps, but before we do that, let's first understand why exactly you need testing in a framework like Django.